بسم الله الرحمن الرحيم

Structural Honesty Verification as a Complementary Static Analysis Class

An Empirical Benchmark of furqan-lint Across 25 Open-Source Python Codebases

Bilal Syed Arfeen
BayyinahEnterprise
November 2026

Abstract

We present an empirical evaluation of furqan-lint, a multi-language static analysis tool that checks for *structural honesty* -- whether a program's surface behavior (what it claims to do) matches its substrate behavior (what it actually does). We benchmark furqan-lint against bandit, the established Python security linter, across 25 widely-used open-source packages totaling 2,327,868 lines of code and 5,252 files. furqan-lint produces more findings than bandit in 22 of 25 packages (median per-package ratio 8.5x). We argue that structural honesty verification represents a complementary analysis class that occupies a distinct region of the finding space -- one that existing linters (style checkers, type checkers, and security scanners) leave unexamined. We provide the full dataset for reproducibility.

Note on terminology. Throughout this paper, finding means a pattern flagged by a tool's rule. Some findings are true positives (genuine problems); others are intentional design choices that happen to match a pattern. The false positive rate is not measured in this study. We evaluate the results as evidence that structural honesty verification occupies a distinct region, not as a claim that every finding represents a defect.

1. Introduction

Static analysis tools for Python fall into well-established categories: style and convention checkers (ruff, flake8, pylint), type checkers (mypy, pyright), and security scanners (bandit, semgrep). Each category targets a specific finding class: formatting inconsistencies, type errors, and known vulnerability patterns, respectively.

We identify a fourth class that existing tools do not systematically address: *structural dishonesty* -- code whose observable interface (variable names, exception handlers, type annotations, API contracts) makes claims that its implementation does not honor. Examples include:

- An **except Exception** block that catches everything but re-raises nothing, claiming to handle errors while silencing them.
- A parameter named **password** assigned the literal string "admin123", claiming secrecy while embedding the secret in source.
- A function accepting ****kwargs** and passing them through to another call without validation, claiming a bounded interface while providing an unbounded one.

These patterns are not style violations, not type errors, and not entries in vulnerability databases. They are gaps between what code says and what code does. We call their detection *structural honesty verification* (SHV).

furqan-lint v2.1.0 implements SHV as 40 rules organized into four families:

- **RING-CLOSE (RC)**: resources that open but do not close (5 rules)
- **MARAD (MR)**: failures that occur but produce no signal (5 rules)
- **TRUST-BOUNDARY (TB)**: security-relevant boundaries that can be crossed (22 active rules + 3 informational, off by default)
- **SCAN-INCOMPLETE (SI)**: regions where analysis has been explicitly abandoned (7 rules)

Three rules (TB-001, TB-002, TB-003) are classified as *informational* and disabled by default because they flag language features (isinstance, deepcopy, type().__name__) rather than likely problems. All benchmark results in this paper use the default configuration, which excludes these three rules.

This paper presents the first large-scale empirical comparison between SHV and conventional security scanning, answering the question: **does structural honesty verification find a meaningfully different set of findings than existing tools, or does it merely duplicate their coverage?**

2. Method

2.1 Package selection

We selected 25 packages from the Python Package Index (PyPI) to maximize diversity across domain (web frameworks, HTTP clients, data science, ORMs, task queues, templating, cryptography, SSH, scraping, CLIs, cloud SDKs, image processing), size (from 6,385 lines to 635,499 lines), and maturity (all widely used in production with active maintenance). No package was excluded based on preliminary results. The selection was made before any scanning was performed.

2.2 Tools compared

Tool	Version	Configuration
furqan-lint	v2.1.0	Default (37 active rules; TB-001/002/003 off)
bandit	1.8.0 (PyPI)	Default (all rules, no exclusions)

2.3 Procedure

Each package was scanned in its installed form from PyPI (site-packages). Both tools were run with default configuration and no rule exclusions. Finding counts were recorded per package. We computed per-package finding counts, the ratio of furqan-lint to bandit findings, finding density (findings per 1,000 lines of code), and severity distribution of furqan-lint findings.

2.4 Threats to validity

Overlap. furqan-lint's rule set includes rules that detect the same patterns bandit does (hardcoded secrets, insecure hashing, etc.). A finding-count comparison therefore includes both overlapping and unique findings. We do not decompose these in this study; a per-finding-pair decomposition across all 25 packages is a target for future work.

Precision. Neither tool's false positive rate is measured. Higher finding counts may reflect broader detection or higher noise. We do not claim that more findings means better analysis; we claim that the findings come from a different region of the analysis space.

Installed vs. repository. Scanning installed packages rather than repository source means test files are typically absent, which affects rules like SI-006 (assert usage) and bandit's B101.

Single bandit version. Results may differ with bandit plugins, alternative configurations, or newer versions.

3. Results

3.1 Aggregate findings

Metric	Value
Packages scanned	25
Total Python files	5,252
Total lines of code	2,327,868
Total furqan-lint findings	39,072
Total bandit findings	29,959
Packages where furqan > bandit	22 of 25
Packages where bandit > furqan	3 of 25

The three packages where bandit produces more findings are numpy, pandas, and scipy. In all three cases, bandit's B101 (assert_used) rule fires heavily on scientific code that uses assert for precondition checking.

3.2 Per-package results

Package	Domain	Files	Lines	furqan	bandit	Ratio
django	Web framework	909	165,634	3,195	281	11.4x
flask	Micro web	24	9,199	135	13	10.4x
fastapi	Async web	52	22,748	859	70	12.3x
tornado	Async net.	76	50,586	1,144	285	4.0x
starlette	ASGI	36	6,947	364	73	5.0x
requests	HTTP lib	19	6,385	223	16	13.9x
httpx	Async HTTP	23	8,828	178	29	6.1x
urllib3	HTTP (low)	36	11,914	189	10	18.9x
aiohttp	Async HTTP	54	28,467	806	178	4.5x
sqlalchemy	SQL ORM	258	253,997	9,771	866	11.3x
celery	Task queue	161	47,496	1,160	49	23.7x
jinja2	Template	25	14,425	363	62	5.9x
pydantic	Validation	105	45,762	2,776	86	32.3x
cryptography	Crypto	78	13,868	374	18	20.8x
paramiko	SSH	42	18,621	252	21	12.0x
scrapy	Scraping	199	38,503	1,716	241	7.1x
click	CLI	17	12,674	174	36	4.8x
rich	Terminal	100	38,515	290	58	5.0x
typer	CLI builder	32	13,911	588	60	9.8x
boto3	AWS SDK	39	8,558	73	0	--
PIL	Image	98	35,149	982	288	3.4x
werkzeug	WSGI	52	21,687	362	28	12.9x
numpy	Numerical	407	243,710	2,069	3,597	0.6x
pandas	Data	1,422	635,499	7,703	17,701	0.4x
scipy	Scientific	988	574,785	3,326	5,893	0.6x

3.3 Per-package ratio distribution

Metric	Value
Mean ratio (n=24, excluding boto3)	9.9x
Median ratio	8.5x
Standard deviation	7.8
Minimum ratio	0.4x (pandas)
Maximum ratio	32.3x (pydantic)

3.4 Severity distribution

furqan-lint assigns one of four severity levels to each finding:

Severity	Count	Percentage
CRITICAL	331	0.8%
HIGH	1,918	4.9%
MEDIUM	17,800	45.6%
LOW	19,023	48.7%
Total	39,072	100%

CRITICAL findings (0.8%) are restricted to patterns with direct security impact: bare except: without re-raise (MR-001), eval()/exec() (TB-005), and buffer overflow functions in C adapters (TB-029). The majority of findings are MEDIUM or LOW severity, consistent with the structural honesty framing: most surface-substrate gaps are quality issues, not security vulnerabilities.

4. Discussion

4.1 Why the ratio varies

The furqan-to-bandit ratio ranges from 0.4x to 32.3x across packages. Three factors explain the variation:

High-ratio packages (pydantic 32.3x, celery 23.7x, cryptography 20.8x, urllib3 18.9x): These packages are well-engineered and have few security vulnerabilities (low bandit count), but furqan-lint's structural checks (kwargs passthrough, error handling patterns, hasattr usage) find substantial room for improvement. This is the core observation: SHV finds patterns that security scanners do not look for, and these patterns are abundant even in well-maintained code.

Medium-ratio packages (django 11.4x, requests 13.9x, werkzeug 12.9x): Both tools find issues, but furqan-lint's broader rule set consistently produces more findings.

Low-ratio packages (numpy 0.6x, pandas 0.4x, scipy 0.6x): These scientific computing packages make extensive use of assert statements. bandit's B101 rule flags every assert, producing thousands of findings in codebases that rely heavily on assert for precondition checking. furqan-lint's SI-006 fires on the same pattern, but the sheer volume of assert usage in scientific code means bandit's total exceeds furqan-lint's in these three packages.

4.2 Complementarity, not superiority

The comparison between furqan-lint and bandit is not a competition to produce more findings. The two tools address different analysis dimensions:

- **bandit** focuses on known security vulnerability patterns: hardcoded passwords, insecure hash algorithms, shell injection, etc.
- **furqan-lint** focuses on structural honesty: whether code's interface matches its implementation, regardless of whether the mismatch has a known CVE.

Many furqan-lint rules have no bandit equivalent (error handling analysis, resource lifecycle, kwargs passthrough, type annotation completeness). Conversely, bandit checks for some import-level patterns that furqan-lint

deliberately omits in favor of usage-level detection. The tools are complementary; a CI pipeline benefits from running both.

4.3 Implications for safety-critical systems

furqan-lint extends beyond Python to five additional languages (C, C++, Java, C#, Ada/SPARK) via tree-sitter-based adapters. The structural honesty concept is language-independent: a catch(...) without rethrow in C++ (AUTOSAR) is the same dishonesty pattern as except Exception: pass in Python, and "when others => null;" in Ada (DO-178C). A single conceptual framework and a single tool covers all six languages, though the non-Python adapters are not benchmarked in this study.

4.4 Limitations

1. Finding count is not defect count. Some findings are intentional design choices (e.g., a deliberately unbounded **kwargs in a plugin system). The false positive rate is not measured in this study and is a critical target for future work.

2. No overlap decomposition. We do not decompose findings into overlapping and unique categories across all 25 packages. A per-finding-pair comparison (matching furqan-lint and bandit findings by file and line) is needed to quantify how much of each tool's output is unique.

3. Installed vs. repository. Scanning installed packages rather than repository source means test files are typically absent, which affects assert-related rules.

4. Single bandit version. Results may differ with bandit plugins, alternative configurations, or newer versions.

5. No ruff baseline. We did not independently verify ruff findings on these packages. The claim that these codebases are convention-clean is not substantiated by this study.

6. Informational rules excluded. Three rules (TB-001/002/003) are disabled by default because they flag language features rather than likely problems. This means the v2.1.0 finding counts are lower than they would be with all 40 rules active.

5. Conclusion

Structural honesty verification, as implemented by furqan-lint v2.1.0, produces more findings than bandit in 22 of 25 packages (median per-package ratio 8.5x). The three exceptions are scientific computing packages where bandit's assert-usage rule fires at high volume.

These results are evidence that structural honesty verification occupies a region of the analysis space that conventional security linters do not cover. The four SHV check families -- RING-CLOSE, MARAD, TRUST-BOUNDARY, and SCAN-INCOMPLETE -- represent a complementary analysis dimension that can be added to any CI pipeline alongside existing tools.

Whether the additional findings represent true defects, or intentional design choices that happen to match structural patterns, is an open empirical question. Measuring precision across a stratified sample is the necessary next step.

The tool is open-source under Apache-2.0 and available at: **https://github.com/BayyinahEnterprise/Kun-Fayakun**

Data availability. The complete benchmark dataset (25 packages, per-package finding counts, severity distributions, tool versions) is included in the supplementary CSV file benchmark_v2.1.0.csv. The benchmark harness script is included as benchmark.py. All code is reproducible from the public repository at tag v2.1.0.

References

[1] B. S. Arfeen, "Structural Honesty Verification: On Surface-Substrate Divergence Across Systems," 2026.

[2] PyCQA, "Bandit: Security linter for Python," https://bandit.readthedocs.io/

[3] C. Marsh et al., "Ruff: An extremely fast Python linter," https://docs.astral.sh/ruff/

[4] OASIS, "Static Analysis Results Interchange Format (SARIF) Version 2.1.0," 2018.

[5] M. Brucker et al., "Tree-sitter: An incremental parsing system for programming tools," https://tree-sitter.github.io/tree-sitter/

بسم الله الرحمن الرحيم